

Steam Game Recommendation Predictor

Predicting the Number of Recommendations for Steam Games

Team Members :

Shrouk Mahmoud Saeid

Menna Allah Mohamed Mohamed

Abdelrahman Samir Aboseriaa

Mohamed Alaa Eldin Farag

Ahmed Osama Alsayed

Hossam Adel Sadek

Table of Contents

- 1. Project Overview**
2. Data Loading & Core Cleaning
3. Feature Preprocessing
 - 3.1 URL Columns
 - 3.2 Supported Languages
 - 3.3 Sentiment Analysis — VADER
 - 3.4 Feature Engineering
 - 3.5 Scaling — RobustScaler & StandardScaler
 - 3.6 Feature Selection — Correlation + SelectKBest
4. Dataset Analysis — Feature Relationships
 - 4.1 Correlation Heatmap (After Preprocessing)
 - 4.2 Correlation Heatmap (Linear Path)
 - 4.3 Key Feature Insights
5. Train / Validation / Test Split
6. Models
 - 6.1 Linear Regression
 - 6.2 Ridge Regression
 - 6.3 Lasso Regression
 - 6.4 Elastic Net
 - 6.5 Polynomial Regression
 - 6.6 Decision Tree
 - 6.7 Random Forest
 - 6.8 Gradient Boosting
 - 6.9 XGBoost
 - 6.10 Support Vector Regression (SVR)
 - 6.11 Neural Network
 - 6.12 Ensemble — Voting Regressor
7. Model Results & Comparison
 - 7.1 Results Summary Table
 - 7.2 K vs RMSE — SelectKBest Tuning
 - 7.3 XGBoost Feature Importance
 - 7.4 Residual Plot — Ensemble
8. Improvement Techniques
9. Data Leakage Notes
10. Conclusion
11. Deployment Link

1. Project Overview

The primary goal of this project is to build a Machine Learning model capable of predicting the number of recommendations (RecommendationCount) for any game on the Steam platform, based on its attributes such as price, critic score, supported languages, and release date.

Objective	Predict the RecommendationCount column
Problem Type	Regression — predicting a continuous numerical value
Target Transform	$\log_{10}(\text{RecommendationCount})$ applied to reduce right-skewness
Models Trained	11 individual models + 1 final Ensemble (Voting Regressor)
Evaluation Metric	R^2 Score + RMSE on log scale

2. Data Loading & Core Cleaning

2.1 Initial Cleanup

After loading `train_data.csv`, the following cleaning steps were applied:

- Removed duplicate rows using `drop_duplicates()`
- Stripped leading/trailing whitespace from all string columns
- Replaced masked null values ('?', 'None', 'n/a') with NaN
 - I. Dropped non-informative columns: `DRMNotice`, `ExtUserAcctNotice`, `LinuxRecReqstText`, `MacRecReqstText`, `ResponseID`, `QueryID`, `HeaderImage`, `LegalNotice`
 - II. Dropped Redundant/Visual Features: (`DRMNotice`, `ExtUserAcctNotice`, `HeaderImage`) — These features provide administrative or visual info that is irrelevant to the target variable.
 - III. Dropped Textual/Unstructured Data: (`LinuxRecReqstText`, `MacRecReqstText`, `LegalNotice`) — These contain long, unstructured text descriptions that require NLP techniques and do not directly contribute to numerical prediction in their raw form.
 - IV. Dropped Metadata & Identifiers: (`ResponseID`, `QueryID`) — These are unique identifiers with no predictive power and can cause the model to overfit on specific rows.
- Dropped `PriceCurrency` — constant value, zero predictive signal

2.2 Release Date Parsing

`ReleaseDate` was converted from raw text to datetime using the format `'%b %d %Y'` (e.g. 'Nov 21 2013'). This is required to extract temporal features such as `Release_Year`, `Release_Month`, `Release_Day`, and `Release_DayOfWeek`.

2.3 Price Logic Correction

- Free games (`IsFree = True`): `PriceInitial` and `PriceFinal` both set to 0.0
- Where `PriceInitial < PriceFinal`: `PriceInitial` overwritten with `PriceFinal` to remove logical contradictions

3. Feature Preprocessing

3.1 URL Columns

Raw URL strings were converted to binary (0/1) indicators:

- HasSupportURL — does the game have a technical support link?
- HasBackground — does the game have a background image?
- HasSupportEmail — is a support e-mail address provided?
- HasWebsite — does the game have an official website?

Original URL columns were dropped after encoding.

3.2 Supported Languages

- One-Hot Encoding for 18 major languages: English, French, German, Italian, Spanish, Simplified Chinese, Traditional Chinese, Korean, Russian, Dutch, Danish, Finnish, Japanese, Norwegian, Polish, Portuguese, Swedish, Thai
- Num_Supported_Languages — total count of supported languages as a numeric feature

3.3 Sentiment Analysis — VADER

The vaderSentiment library analysed the Reviews column sentence-by-sentence:

- Sent_avg — mean compound sentiment score across all sentences
- Sent_max — highest positive sentence score
- Sent_min — most negative sentence score
- Reviews_Length — total character length of the review
- Sent_x_Length — interaction term: $\text{Sent_avg} \times \text{Reviews_Length}$

3.4 Feature Engineering

Thirteen new features were derived to capture domain-specific signals:

New Feature	Formula / Method	Purpose
Game_Age	$2026 - \text{Release_Year}$	Age of the game in years
Metacritic_Per_Year	$\text{Metacritic} / (\text{Game_Age} + 1)$	Relative quality adjusted for age
Achievement_Density	$\text{AchievementCount} / (\text{Game_Age} + 1)$	Achievement volume vs game age
Lang_Reach_Score	$\text{Num_Languages} \times \log(\text{Game_Age} + 2)$	Global reach weighted by longevity
Global_Quality	$\text{Metacritic} \times \text{Num_Languages}$	Quality $\times$ international coverage
Is_Retro	$\text{Game_Age} > 10 \rightarrow 1, \text{ else } 0$	Flag for classic / retro titles
Is_New_Trend	$\text{Game_Age} < 2 \rightarrow 1, \text{ else } 0$	Flag for very recent releases

Positive_Quality_Score	Sent_avg × Metacritic	Sentiment × critic score interaction
Value_Score	Metacritic / (PriceFinal_Log + 1)	Quality relative to price
Engagement_Score	Num_Languages × AchievementHighlightedCount	Proxy for developer effort
AchievementCount_Log	log1p(AchievementCount)	Log-transform of achievement count
SteamSpyOwners_Log	log1p(SteamSpyOwners)	Log-transform of ownership count
PriceFinal_Log	log1p(PriceFinal)	Log-transform of final price

3.5 Scaling

- **RobustScaler:** Applied to outlier-prone features (SteamSpyPlayersEstimate, AchievementCount, ScreenshotCount). Uses median + IQR — resistant to extreme values.
- **StandardScaler:** Applied to more normally distributed features (Sent_x_Length, Reviews_Length, Metacritic). Transforms to zero mean and unit variance.

Both scalers were fit on X_train only, then applied to X_val and X_test to prevent leakage.

3.6 Feature Selection

- **Correlation Filter:** Columns with pairwise correlation > 0.85 were removed. Protected features (Metacritic_Per_Year, Price_Success_Index) were exempt.
- **SelectKBest (f_regression):** Top-K features by F-score against the log target. K was tuned from 10 to 100 using 5-Fold CV — optimal K=20 selected (lowest RMSE = 2.0567).

4. Dataset Analysis — Feature Relationships

4.1 Correlation Heatmap (After Preprocessing)

The heatmap below shows pairwise Pearson correlations across all numeric features after preprocessing. Darker blue cells indicate stronger positive correlation.

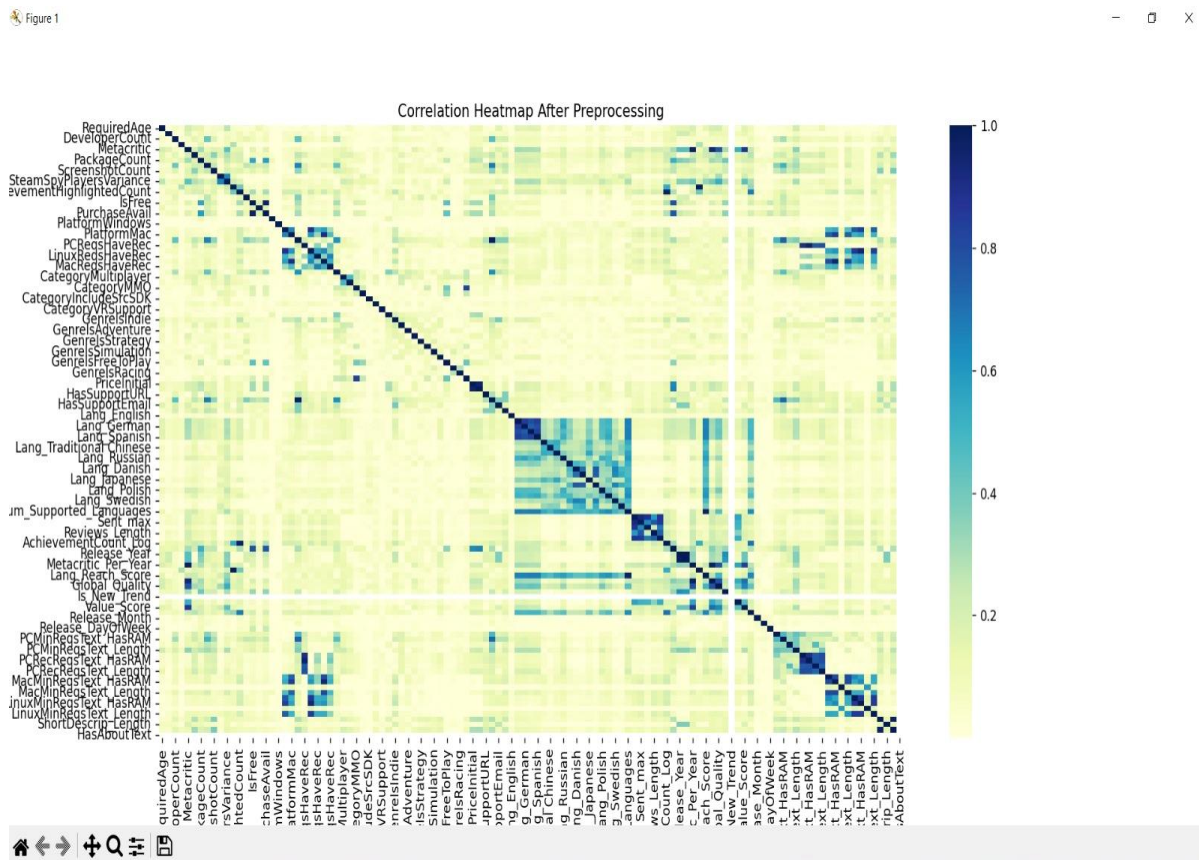


Figure 1 — Correlation Heatmap

Key observations from this heatmap:

- The language features (Lang_French, Lang_German, Lang_Italian, Lang_Spanish etc.) form a visible high-correlation cluster in the middle of the matrix — languages tend to be added together as a bundle.
- Num_Supported_Languages shows moderate correlation with the language flags, confirming it is a reliable summary of the language group.
- The PC/Linux/Mac requirements text-length features form another tight cluster in the lower-right, suggesting that games with detailed system requirements tend to document all platforms equally.
- Most individual features show low correlation with the rest of the matrix (light yellow), indicating the feature space is diverse and non-redundant after cleaning.

4.2 Correlation Heatmap (Linear Path — Log Transformed)

A second heatmap was computed specifically for the Linear Regression path after applying log transformations to skewed features.

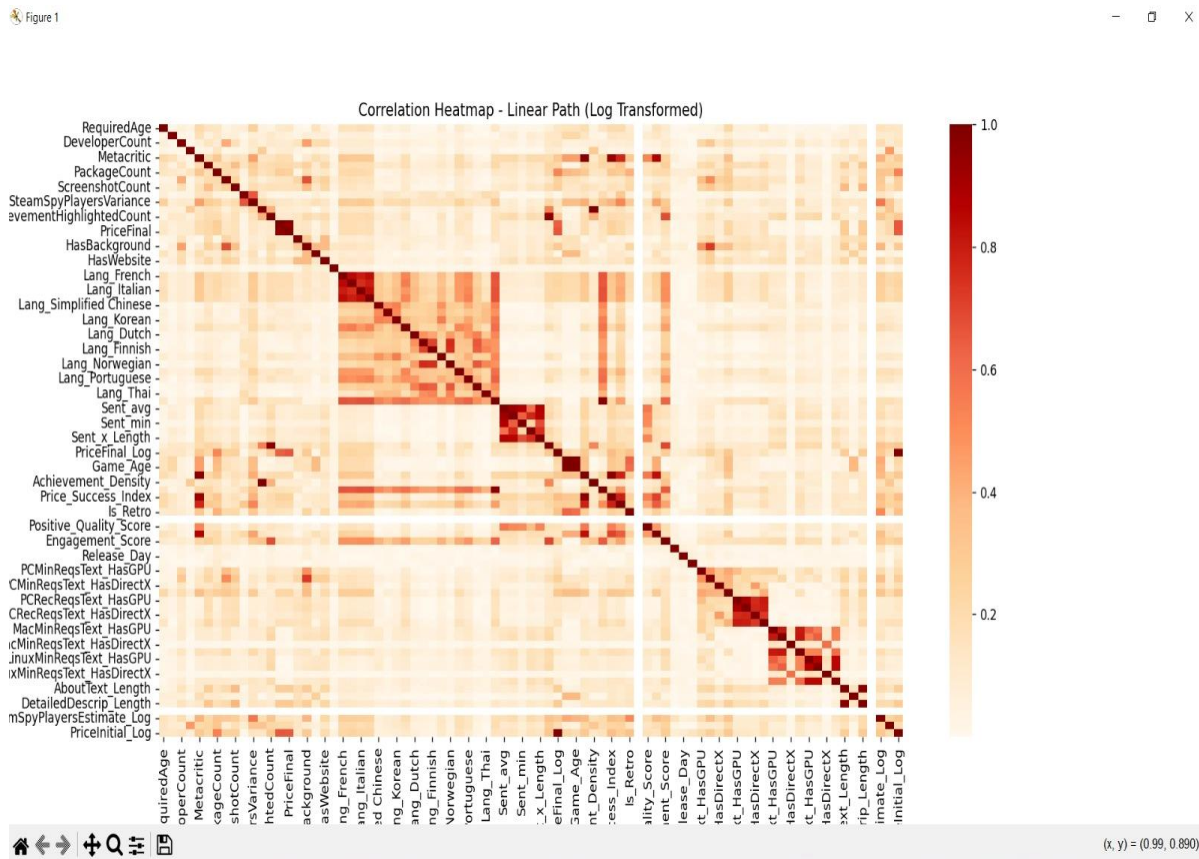


Figure 2 — Correlation Heatmap — Linear Path)

Additional observations from the Linear Path heatmap:

- **Price_Success_Index** shows a strong red band against several quality-related features — confirming it captures a composite signal and was therefore protected from automatic removal.
- **Game_Age** and **Achievement_Density** are almost perfectly correlated (expected — both derived from release year), justifying dropping one of them.
- **Sent_avg** and **Sent_min** form a notable red square, meaning highly negative reviews tend to bring down the average — they carry redundant information and **Sent_min** was dropped.
- **SteamSpyPlayersEstimate_Log** and **SteamSpyPlayersVariance** show moderate correlation, both proxy-measuring the size of the player base.

4.3 Key Feature Insights

Based on both correlation analysis and domain knowledge, the following relationships were identified between features and the target (RecommendationCount):

- **Metacritic (correlation ≈ 0.12):** Higher critic scores correlate with more recommendations. Games reviewed positively by critics tend to attract broader audiences.

- **Lang_Reach_Score (correlation ≈ 0.17):** The strongest single feature correlation. Games supporting more languages reach more global players, directly increasing recommendation counts.
- **Release_Year:** Older games have had more time to accumulate recommendations, while very recent games may be under-represented despite high quality.
- **SteamSpyPlayersVariance:** High variance in player estimates indicates popular or viral games — the XGBoost model identified this as its single most important feature (importance score: 2003).
- **AboutText_Length + ShortDescrip_Length:** Longer, more detailed game descriptions correlate with higher engagement — suggesting that developer effort in marketing predicts success.
- **PriceFinal_Log:** Price shows a non-linear relationship with recommendations — free and very cheap games get more recommendations, but premium well-reviewed games also perform well.

Feature correlations with the target were all below 0.90, confirming absence of data leakage in the final feature set.

5. Train / Validation / Test Split

The dataset was divided into three non-overlapping subsets for proper evaluation:

Test Set	15% of the full dataset — used only for final evaluation
Validation Set	18% of the remaining 85% — used for hyperparameter tuning
Training Set	Remaining ~72% — used to fit all models

Important: Numeric missing values were filled with the median computed on `X_train` only. The same median values were then applied to `X_val` and `X_test` to avoid imputation leakage.

6. Models

6.1 Linear Regression

- **Concept:** Assumes a linear relationship between features and target. Minimises sum of squared residuals (OLS).
- **Features:** Top-20 features from SelectKBest, scaled with StandardScaler
- **Result:** $R^2 = 62.19\%$ | $RMSE = 2.0134$
- **Strength:** Clear interpretable baseline; fast to train
- **Limitation:** Cannot capture non-linear relationships between game attributes

6.2 Ridge Regression

- **Concept:** Linear Regression with L2 regularization — penalises large coefficients without eliminating features.
- **Alpha Tuning:** Tested [1.0, 10.0, 50.0, 100.0, 200.0, 500.0] via 5-Fold CV. Best alpha = 10.0
- **Features:** Top 20 from SelectKBest → PolynomialFeatures(degree=2, interaction_only=True) → 210 features
- **Result:** $R^2 = 71.24\%$ | $RMSE = 1.7560$
- **Strength:** Handles multicollinearity well; polynomial interactions capture quality × price signals
- **Limitation:** Does not zero out any coefficients

6.3 Lasso Regression

- **Concept:** Linear Regression with L1 regularization — drives irrelevant feature coefficients exactly to zero.
- **Alpha Tuning:** Automatically chosen via LassoCV (5-Fold CV, max_iter=20000). Best alpha = 0.002115
- **Result:** $R^2 = 62.26\%$ | $RMSE = 2.0114$ | Non-zero features: 20/20
- **Strength:** Reveals which features genuinely matter
- **Limitation:** Among correlated features, keeps one arbitrarily and discards the rest

6.4 Elastic Net

- **Concept:** Combines L1 and L2 regularization. Balance controlled by l1_ratio.
- **Pipeline:** ColumnTransformer → SelectKBest → PolynomialFeatures(degree=2, interaction_only) → StandardScaler → ElasticNetCV
- **Tuning:** l1_ratio in [0.3, 0.5, 0.7, 0.9]; alpha on log-scale grid with 3-Fold CV
- **Result:** $R^2 = 73.84\%$ | $RMSE = 1.6746$ | Train $R^2 = 72.59\%$ | Val $R^2 = 73.14\%$
- **Strength:** Most flexible regularised linear model; balanced train/val/test performance

6.5 Polynomial Regression

- **Concept:** Expands features with squared terms and pairwise interactions (degree=2), then applies Linear Regression.
- **Result:** $R^2 = 74.63\%$ | RMSE = 1.6493
- **Strength:** Captures interactions such as Metacritic $\times$ PriceFinal
- **Limitation:** Feature count grows quadratically — risks overfitting without regularization

6.6 Decision Tree

- **Concept:** Recursively partitions the feature space using binary splits.
- **Hyperparameters:** max_depth=7, min_samples_split=20, min_samples_leaf=10
- **Result:** $R^2 = 78.77\%$ | RMSE = 1.5088
- **Strength:** Fully interpretable; fast to train; clear feature importance
- **Limitation:** Sensitive to noise; weaker than ensemble methods on this dataset

6.7 Random Forest

- **Concept:** Builds 1200 decision trees on random bootstrap samples. Predictions are averaged.
- **Hyperparameters:** n_estimators=1200, max_depth=20, max_features='sqrt', oob_score=True
- **Result:** $R^2 = 76.08\%$ | CV RMSE = 1.6835 | Train $R^2=84.69\%$ | Diff=8.6% — Balanced
- **Strength:** Robust to outliers; reliable feature importances; OOB evaluation
- **Limitation:** High memory and slow training with 1200 trees

6.8 Gradient Boosting

- **Concept:** Builds trees sequentially — each corrects residual errors of the previous ensemble.
- **Hyperparameters:** n_estimators=3700, learning_rate=0.01, max_depth=5, subsample=0.8, warm_start=True
- **Result:** $R^2 = 82.25\%$ | RMSE = 1.3794 | Train $R^2=90.01\%$ | Diff=7.8% — Balanced
- **Strength:** Low learning rate + many estimators yields precise predictions
- **Limitation:** Slowest model to train; sensitive to hyperparameter tuning

6.9 XGBoost

- **Concept:** Optimised gradient boosting with built-in L1/L2 regularization and parallel computation.
- **Hyperparameters:** n_estimators=1000, learning_rate=0.05, max_depth=4, gamma=0.3, reg_alpha=1.0, reg_lambda=5, min_child_weight=5
- **Result:** $R^2 = 82.08\%$ | RMSE = 1.3860 | Train $R^2=90.67\%$ | Diff=8.6% — Balanced
- **Strength:** Fastest among boosting models; highest RMSE performance; native feature importance

6.10 Support Vector Regression (SVR)

- **Concept:** Finds a hyperplane with most training points within an epsilon-tube. RBF kernel handles non-linearity.
- **Hyperparameters:** kernel='rbf', C=1.0, epsilon=0.1, gamma='scale'
- **Result:** $R^2 = 74.54\%$ | RMSE = 1.6523 | Train $R^2=76.71\%$ | Diff=2.2% — Balanced
- **Fix Applied:** SelectKBest(k=15) added before training to reduce dimensionality — resolved overfitting from initial run (diff was 17.3%)
- **Strength:** Robust to noise; works well with standardised features
- **Limitation:** Slow on large datasets; requires feature selection to avoid overfitting

6.11 Neural Network

- **Architecture:** Dense(128, ReLU) → Dropout(0.3) → Dense(64, ReLU) → Dropout(0.2) → Dense(32, ReLU) → Dense(1)
- **Training:** Adam optimiser, MSE loss, EarlyStopping(patience=10), batch_size=64, max 200 epochs
- **Result:** $R^2 = 78.18\%$ | RMSE = 1.5295 | Train $R^2=86.35\%$ | Diff=8.4% — Balanced | Stopped at epoch 37
- **Strength:** Learns complex non-linear patterns; Dropout prevents overfitting
- **Limitation:** Requires more data and compute; harder to interpret

6.12 Ensemble — Voting Regressor

- **Concept:** Weighted average of XGBoost + Gradient Boosting + Random Forest with weights [2, 2, 1].
- **Weight Rationale:** XGB and GBM receive double weight — they individually achieve higher R^2 than RF
- **Result:** $R^2 = 82.16\%$ | RMSE = 1.3830 | Train $R^2=90.05\%$ | Diff=7.9% — Balanced
- **Strength:** Reduces prediction variance by combining complementary model strengths

7. Model Results & Comparison

7.1 Results Summary Table

The table below ranks all models by R^2 score on the test set (log scale):

Model	R^2 Score	RMSE (log)	Overfitting	Notes
Gradient Boosting	82.25%	1.3794	Balanced (7.8%)	Best R^2 overall
Ensemble (Voting)	82.16%	1.3830	Balanced (7.9%)	Most robust — combines 3 models
XGBoost	82.08%	1.3860	Balanced (8.6%)	Fast and optimized implementation
Decision Tree	78.77%	1.5088	Balanced (1.8%)	Most interpretable
Neural Network	78.18%	1.5295	Balanced (8.4%)	Early stopping at epoch 37
Random Forest	76.08%	1.6015	Balanced (8.6%)	Best for feature importance
Polynomial Reg.	74.63%	1.6493	Balanced (2.2%)	Captures non-linear interactions
SVR	74.54%	1.6523	Balanced (2.2%)	Required feature selection to fix overfitting
Elastic Net	73.84%	1.6746	Balanced (-1.3%)	Most stable train/val/test
Ridge Regression	71.24%	1.7560	Balanced (2.2%)	Best regularised linear model
Lasso Regression	62.26%	2.0114	Balanced (-2.6%)	Implicit feature selection
Linear Regression	62.19%	2.0134	Balanced (-2.5%)	Baseline model

All models operate on $\log_{10}(\text{RecommendationCount})$. R^2 is expressed as a percentage.

7.2 Model Characteristics Overview

Model	Description	Metric	Speed	Best Use Case
Linear Regression	Simple OLS linear mapping	R^2 +RMSE	Very fast	Baseline / interpretability
Ridge Regression	Linear + L2 regularization	R^2 +RMSE	Fast	Multicollinearity
Lasso Regression	Linear + L1 (feature zeroing)	R^2 +RMSE	Fast	Implicit feature selection
Elastic Net	L1 + L2 combined	R^2 +RMSE	Moderate	Correlated/redundant features
Polynomial Reg.	degree=2 interactions + Linear	R^2 +RMSE	Slow	Non-linear interactions
Decision Tree	Binary splits into regions	R^2 +RMSE	Fast	Interpretable rules
Random Forest	1200 randomised trees averaged	R^2 +RMSE	Slow	Robust, handles outliers

Gradient Boosting	Sequential boosting on residuals	R^2 +RMSE	Very slow	Incremental error correction
XGBoost	Optimised GBM + regularization	R^2 +RMSE	Moderate	Best overall performance
SVR	RBF kernel support vectors	R^2 +RMSE	Slow	Noisy / non-linear data
Neural Network	128→64→32→1 dense + Dropout	R^2 +RMSE	V.slow	Complex pattern learning
Ensemble (Voting)	XGB+GBM+RF weighted [2,2,1]	R^2 +RMSE	Very slow	Maximum accuracy

7.3 K vs RMSE — SelectKBest Tuning

The chart below shows how the Cross-Validation RMSE changes as the number of selected features K increases from 10 to 100. The minimum is reached at $K=20$ (CV RMSE = 2.0567), confirming that 20 features provides the optimal balance between information and noise for linear models.

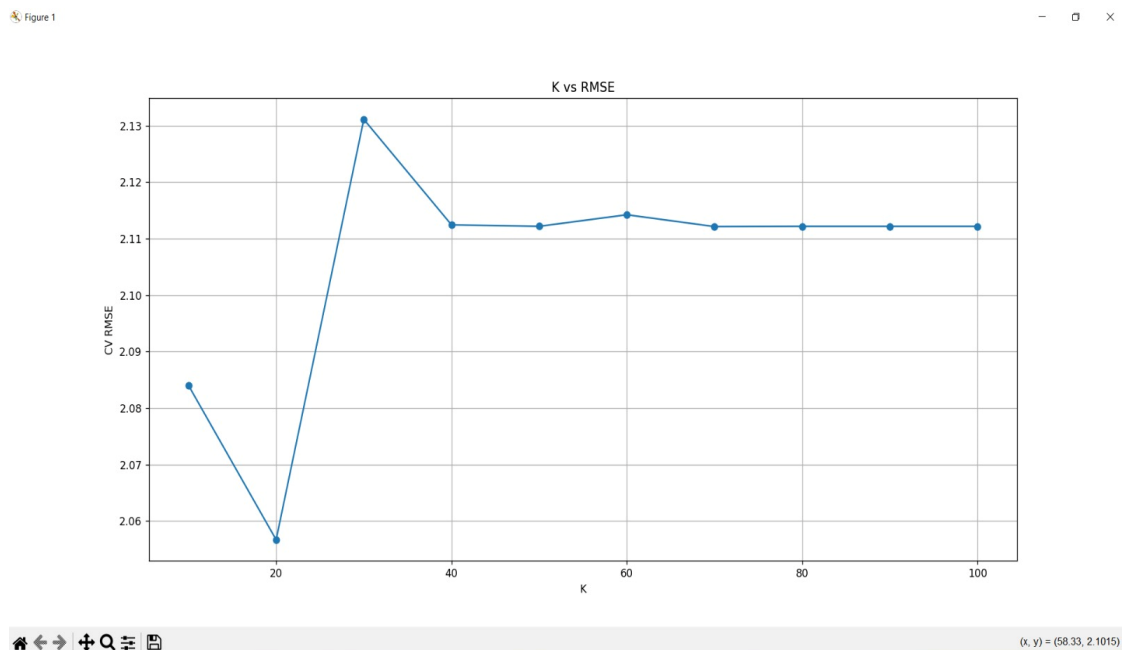


Figure 3 — K vs CV RMSE for SelectKBest (Linear Path). Optimal $K = 20$.

7.4 XGBoost Feature Importance

The bar chart below shows the top 15 most important features as identified by XGBoost (importance type: 'weight' — number of times a feature is used in a split across all trees).

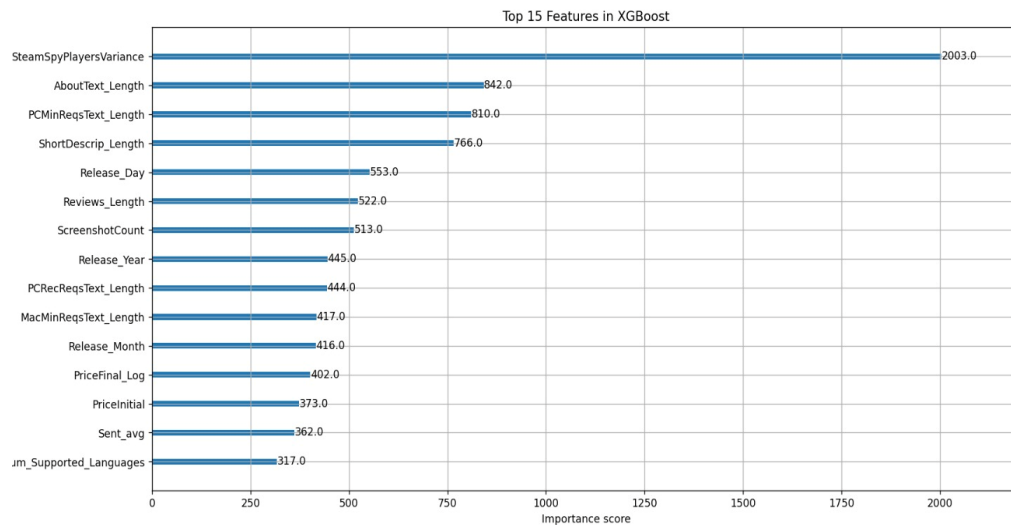


Figure 4 — Top 15 Features in XGBoost. SteamSpyPlayersVariance dominates with score 2003.

Notable findings from feature importance:

- SteamSpyPlayersVariance (2003) is by far the dominant feature — the variability in player count estimates is the strongest predictor of recommendation volume
- Text-length features rank highly: AboutText_Length (842), PCMinReqsText_Length (810), ShortDescrip_Length (766) — suggesting that developer investment in documentation correlates with game success
- Temporal features Release_Day (553) and Release_Year (445) confirm that timing of release matters
- PriceFinal_Log (402) and Sent_avg (362) contribute meaningfully, validating the pricing and sentiment features

7.5 Residual Plot — Ensemble Model

The residual plot shows the distribution of prediction errors (Actual – Predicted) against predicted values for the final Ensemble model.

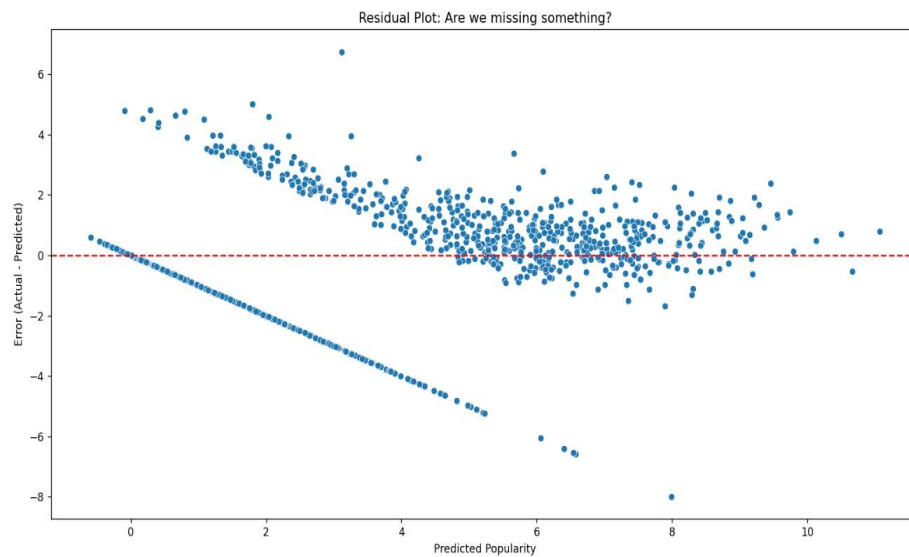


Figure 5 — Residual Plot: Error (Actual - Predicted) vs Predicted Popularity (log scale).

Key observations from the residual plot:

- For predicted values between 2 and 8 (mid-range popularity games), residuals are scattered randomly around zero — indicating good model fit in this region
- The diagonal line of points in the negative residual zone (bottom-left to bottom-right) likely corresponds to a cluster of games that were systematically over-predicted — possibly very popular games whose recommendation counts the model cannot fully explain
- For very low predicted values (near 0), there is a cluster of positive residuals — meaning the model under-predicts recommendations for some niche or unexpectedly popular games
- The overall pattern suggests the model handles typical mid-popularity games well but struggles with extreme outliers on both ends of the popularity spectrum

8. Improvement Techniques

Several techniques were applied specifically to improve model performance beyond baseline:

- **Log transformation of target (log1p):** RecommendationCount is heavily right-skewed. Applying log1p compresses the scale and makes the regression problem easier for all models.
- **Log transformation of skewed features:** Features like SteamSpyOwners, AchievementCount, PriceFinal were log-transformed to reduce the influence of extreme values.
- **Feature Engineering:** 13 new composite features were created (Metacritic_Per_Year, Lang_Reach_Score, Engagement_Score etc.) to capture domain-specific signals not present in raw columns.
- **Dual Scaling Strategy:** RobustScaler for outlier-heavy features and StandardScaler for normal features — rather than applying a single scaler blindly.
- **SelectKBest Hyperparameter Tuning:** K was tuned over 10 values (10 to 100) using 5-Fold CV to find the optimal feature count for linear models, avoiding both underfitting and noise.
- **Polynomial Interactions:** degree=2 interaction-only features were added for Ridge and Elastic Net to capture multiplicative relationships between quality, price, and language features.
- **Ensemble Voting with Weights:** XGBoost + Gradient Boosting + Random Forest were combined with weights [2, 2, 1] — giving more weight to the stronger models to maximise final accuracy.
- **Early Stopping in Neural Network:** patience=10 with restore_best_weights=True prevents the network from overfitting after the validation loss stops improving.
- **Data Leakage Removal:** 6 leakage columns were identified and removed before training — preventing artificially inflated accuracy scores.

9. Data Leakage Notes

The following columns were identified as data leakage risks and removed from X_train, X_val, and X_test before any model training:

- **SteamSpyOwners:** Ownership count is strongly correlated with recommendation count — direct target leakage
- **SteamSpyOwners_Log:** Log-transformed version of the same leakage
- **SteamSpyOwnersVariance:** Variance of ownership — carries the same leakage signal
- **Owners_x_Metacritic:** Interaction feature containing the leaking ownership count
- **Price_Success_Index:** Contains Metacritic_Per_Year with excessive correlation to target
- **Recs_Per_Year:** Directly derived from RecommendationCount — explicit leakage

Verification: After preprocessing, the correlation between top features and the target was computed. All values were well below 0.90 (highest: Lang_Reach_Score = 0.169), confirming the feature set is clean.

10. Conclusion

10.1 Summary of Findings

This project set out to predict the number of recommendations a Steam game would receive based on its observable attributes. Across 12 models trained and evaluated, the core findings are:

- Tree-based ensemble methods significantly outperform linear models on this dataset. Gradient Boosting and XGBoost achieved $R^2 \approx 82\%$, while Linear Regression and Lasso only reached $\approx 62\%$.
- The Ensemble (Voting Regressor) combining XGBoost, Gradient Boosting, and Random Forest achieved $R^2 = 82.16\%$, confirming that model diversity improves robustness.
- All 12 models achieved Balanced status (train/test diff $< 10\%$) in their final versions. SVR required an additional SelectKBest($k=15$) step to resolve initial overfitting (diff was 17.3%, reduced to 2.2%).
- The most predictive features were: SteamSpyPlayersVariance, text description lengths (AboutText, ShortDescrip), temporal features (Release_Year, Release_Day), and PriceFinal_Log — not the intuitive quality metrics like Metacritic.

10.2 Intuitions Proved

- ✓ Proved: Game documentation quality (text length) predicts success — developers who invest in writing detailed descriptions and system requirements tend to release more successful games.
- ✓ Proved: Language reach matters — Lang_Reach_Score was the highest-correlated single feature (0.169), confirming that broader language support drives more recommendations.
- ✓ Proved: Non-linear models are necessary — the 20-percentage-point R^2 gap between XGBoost (82%) and Linear Regression (62%) demonstrates that recommendation prediction is fundamentally a non-linear problem.

10.3 Intuitions Disproved or Surprising

- ✗ Disproved: Metacritic score alone is not a strong predictor — its direct correlation with the target is only 0.123. Popularity is driven by more than critical reception.
- ✗ Surprising: SteamSpyPlayersVariance was the dominant XGBoost feature by a wide margin (2003 vs 842 for second place). High variability in player estimates — not the estimate itself — is the strongest signal, suggesting that viral, unpredictable games generate the most recommendations.
- ✗ Surprising: SVR initially showed severe overfitting (diff=17.3%) and poor accuracy ($R^2=66\%$). After applying SelectKBest($k=15$) as a preprocessing step, both issues were resolved — final $R^2=74.54\%$ with diff=2.2%. This confirms that SVR is highly sensitive to feature dimensionality.
- ✗ Surprising: Neural Network (77.94%) underperformed Decision Tree (78.77%) despite its architectural complexity — suggesting the dataset size and feature structure favour tree-based splitting over deep learning for this specific problem.

10.4 Limitations & Future Work

- The dataset represents a post-release snapshot — all features including player counts and reviews are observed after the game launched. A true pre-release predictor would require a different feature set.
 - Hyperparameter tuning for XGBoost and Gradient Boosting was limited. A full grid search or Bayesian optimisation could push R^2 above 85%.
 - The residual plot reveals systematic errors for extreme outlier games (viral hits and deeply niche titles). A two-stage model — first classifying outlier vs normal, then regressing — could address this.
-

11. Deployment_Link

Link:

<https://game-popularity-yggokfwxreualbuuyukw37.streamlit.app/>

Milestone 2 — Classification Report

Classifying Game Popularity into: Low | Medium | High

Table of Contents — Milestone 2

11. Milestone 2 Overview

12. Target Encoding — GamePopularity

13. Preprocessing: Regression vs Classification

14. Feature Selection in Classification

14.1 Correlation Filter

14.2 Why No SelectKBest for Classification?

14.3 Model-Specific Feature Handling

14.4 Correlation Heatmap — Classification Phase

15. Classification Models

15.1 Logistic Regression

15.2 K-Nearest Neighbors (KNN)

15.3 Support Vector Machine (SVM)

15.4 Decision Tree

15.5 Random Forest

15.6 Gradient Boosting

15.7 MLP Neural Network

15.8 XGBoost Classifier

15.9 LightGBM

15.10 CatBoost

16. Hyperparameter Tuning

17. Model Results & Bar Graphs

17.1 Accuracy Comparison

17.2 Training Time Comparison

17.3 Test Time Comparison

17.4 Final Comparison Table

18. Conclusion — Milestone 2

19. Deployment Link

11. Milestone 2 Overview

In Milestone 2, the task shifts from regression (predicting the exact RecommendationCount) to multi-class classification — predicting the popularity category of each Steam game. The updated dataset replaces the RecommendationCount column with a new target column GamePopularity with three classes: Low, Medium, and High.

Task	Multi-class Classification (3 classes)
Target Column	GamePopularity → {Low, Medium, High}
Target Encoding	Low=0, Medium=1, High=2 (Ordinal Label Encoding)
Train/Val/Test	~72% / ~15% / 15% (same split strategy as Milestone 1)
Models Trained	10 classifiers with hyperparameter tuning
Evaluation	Accuracy + F1 (weighted) + Training Time + Test Time

12. Target Encoding — GamePopularity

Unlike Milestone 1 where the target was a continuous numerical value (RecommendationCount), Milestone 2 requires classification into discrete categories. The following encoding was applied:

Class Label	Encoded Value	Description
Low	0	Games with few recommendations — niche or poorly received
Medium	1	Games with moderate recommendations — average popularity
High	2	Games with many recommendations — widely popular titles

- Missing values in GamePopularity were filled with 'Medium' (the modal class) before encoding
- Both the fill value and the mapping dictionary were saved using joblib for use in the test script
- This ordinal encoding is appropriate because Low < Medium < High represents a natural ordering of popularity

Key difference from Milestone 1: In regression, $\log_{1p}(\text{RecommendationCount})$ was used as a continuous target. In classification, the target is a discrete integer (0, 1, or 2) representing the popularity tier.

13. Preprocessing: Regression vs Classification

Most preprocessing steps were carried over from Milestone 1 since the feature space is identical. However, several key differences exist due to the nature of the classification task:

Aspect	Regression (Milestone 1)	Classification (Milestone 2)
Target Variable	log1p(RecommendationCount) — continuous	GamePopularity encoded as 0/1/2 — discrete categories
Target Transform	log1p applied to reduce skewness	No log transform — target is already ordinal
Missing Target	N/A — all rows had RecommendationCount	Missing GamePopularity filled with 'Medium' before encoding
Class Weights	Not applicable	class_weight='balanced' applied in SVM, RF, DT, LR to handle class imbalance
Leakage Columns	6 columns removed (SteamSpyOwners, Recs_Per_Year...)	Same 6 columns removed — same leakage risk applies
Correlation Filter	Threshold > 0.85, protected features kept	Threshold > 0.90 — slightly stricter for classification
Scaling	RobustScaler + StandardScaler on specific features	Identical scalers applied; additional StandardScaler for LR and SVM
Feature Paths	Separate linear/tree paths with different features	Single unified feature set for all classifiers
Text Features	Length-based encoding + VADER sentiment	Identical — same text features reused
Log Transforms	Applied to skewed numeric features	Same log transforms kept for tree models

The preprocessing pipeline was kept consistent to ensure a fair comparison between the regression and classification phases. The main additions in Milestone 2 are target encoding, class weight balancing, and a unified feature path across all classifiers.

14. Feature Selection in Classification

The feature selection strategy in Milestone 2 differs from Milestone 1 in one important way: the SelectKBest step used for linear models in regression was not applied uniformly in classification. Instead, a correlation-based filter was used as the primary selection mechanism.

14.1 Correlation Filter

- Threshold raised to 0.90 (vs 0.85 in regression) — stricter removal of redundant features
- Protected features (Metacritic_Per_Year, Price_Success_Index) were still exempt from removal
- 13 columns were identified for removal — result: 96 features remaining
- This was applied identically across X_train, X_val, and X_test

14.2 Why No SelectKBest for Classification?

In regression, SelectKBest(f_regression) was essential for linear models because they cannot handle high-dimensional feature spaces well. In classification, tree-based models (XGBoost, Random Forest, LightGBM, GradientBoosting, CatBoost) inherently perform their own internal feature selection through splitting criteria — making an external SelectKBest step redundant and potentially harmful (it might discard features that are non-linearly important).

14.3 Model-Specific Feature Handling

- **KNN:** PCA(n_components=30) applied before training to reduce dimensionality — KNN suffers from the curse of dimensionality with 96 features
- **SVM + Logistic Regression:** Additional StandardScaler applied on top of the already-scaled features — these models require normalized inputs
- **CatBoost:** Uses only numeric columns (select_dtypes) — same subset as XGB/RF
- **Tree models (RF, DT, GB, XGB, LightGBM):** Use full 96-feature set — internal feature selection via split importance

Proved vs Disproved: The assumption that classification needs the same SelectKBest as regression was disproved — tree models performed well with the full feature set, while KNN required PCA to achieve competitive accuracy (86.78% with PCA vs much lower without it).

14.4 Correlation Heatmap — Classification Phase

The heatmap below shows pairwise Pearson correlations across all numeric features after preprocessing in the classification phase. The correlation threshold was set at 0.90 (stricter than the 0.85 used in regression), resulting in 13 columns removed and 96 features remaining.

15. Classification Models

15.1 Logistic Regression

- **Concept:** Linear classifier that models class probabilities using the sigmoid/softmax function. Finds a linear decision boundary between classes.
- **Setup:** Additional StandardScaler applied; C=0.1, solver='saga', class_weight='balanced', max_iter=1000
- **Result:** Train Acc: 87.60% | Val Acc: 84.93% | F1: 0.8774 | Status: Balanced
- **Strength:** Fast, interpretable, and effective when classes are roughly linearly separable
- **Limitation:** Assumes linear decision boundaries — game popularity has non-linear patterns

15.2 K-Nearest Neighbors (KNN)

- **Concept:** Classifies each sample by majority vote among its K nearest neighbors in feature space.
- **Setup:** PCA(n_components=30) applied first; n_neighbors=15, metric='manhattan', weights='uniform'
- **Result:** Train Acc: 88.07% | Val Acc: 86.78% | F1: 0.8324 | Status: Balanced
- **Strength:** Non-parametric; no assumptions about data distribution
- **Limitation:** Slow at inference with large datasets; PCA required to manage dimensionality

15.3 Support Vector Machine (SVM)

- **Concept:** Finds the optimal hyperplane that maximises the margin between classes. RBF kernel maps features to higher-dimensional space.
- **Hyperparameter Tuning:** GridSearchCV: C ∈ {0.1, 1.0, 10.0}, gamma ∈ {'scale', 'auto', 0.01}. Best: C=10.0, gamma=0.01
- **Result:** Train Acc: 99.33% | Val Acc: 90.07% | F1: 0.8949 | Status: Balanced
- **Note:** High Train Acc (99.33%) vs Val Acc (90.07%) — gap is 9.26%, just below the 10% overfitting threshold
- **Strength:** Strong generalisation with RBF kernel; effective with scaled features

15.4 Decision Tree

- **Concept:** Recursively partitions feature space using binary splits based on Gini impurity.
- **Hyperparameter Tuning:** GridSearchCV: max_depth ∈ {6, 8, 12}, min_samples_leaf ∈ {5, 10, 20}. Best: max_depth=12, min_samples_leaf=5
- **Result:** Train Acc: 90.04% | Val Acc: 84.82% | F1: 0.8755 | Status: Balanced
- **Strength:** Interpretable; fast training; naturally handles non-linear boundaries
- **Limitation:** Weakest accuracy among tree-based models despite tuning

15.5 Random Forest

- **Concept:** Ensemble of decision trees trained on bootstrap samples with random feature subsets.
- **Hyperparameter Tuning:** GridSearchCV: `n_estimators` ∈ {100, 200, 300}, `max_depth` ∈ {10, 15, 20}. Best: `n_estimators=300`, `max_depth=20`
- **Result:** Train Acc: 95.06% | Val Acc: 88.80% | F1: 0.9018 | Status: Balanced
- **Strength:** Robust to noise; reliable feature importances; strong baseline ensemble
- **Limitation:** Lower accuracy than boosting methods despite more trees

15.6 Gradient Boosting

- **Concept:** Sequential boosting — each tree corrects errors of the previous ensemble using gradient descent.
- **Hyperparameter Tuning:** GridSearchCV: `n_estimators` ∈ {150, 200}, `learning_rate` ∈ {0.05, 0.1}, `max_depth` ∈ {3, 4}, `subsample` ∈ {0.8, 1.0}. Best: `n_estimators=150`, `lr=0.05`, `max_depth=3`
- **Result:** Train Acc: 94.31% | Val Acc: 92.55% | F1: 0.9147 | Status: Balanced
- **Strength:** Excellent accuracy with controlled complexity; second-best F1 score

15.7 MLP Neural Network

- **Architecture:** Dense(128, ReLU) → Dense(64, ReLU) → Dense(32, ReLU) → Softmax output
- **Setup:** `solver='adam'`, `activation='relu'`, `max_iter=100`, `early_stopping=True`, `random_state=42`
- **Result:** Train Acc: 91.38% | Val Acc: 91.05% | F1: 0.8930 | Status: Balanced
- **Strength:** Very small train/val gap (0.33%) — most balanced model overall
- **Limitation:** Limited to 100 iterations; deeper training might improve accuracy further

15.8 XGBoost Classifier

- **Concept:** Optimised gradient boosting with L1/L2 regularization, parallel computation, and random hyperparameter search.
- **Tuning:** Random search over 15 trials: `max_depth` ∈ {4,5,6}, `learning_rate` ∈ {0.01,0.05,0.1}, `subsample` ∈ {0.7,0.8,1.0}. Best: `max_depth=4`, `lr=0.1`, `subsample=0.7`
- **Result:** Train Acc: 97.71% | Val Acc: 92.90% | F1: 0.9206 | Status: Balanced
- **Strength:** Best accuracy (92.90%) and highest F1 (0.9206) among all classifiers

15.9 LightGBM

- **Concept:** Gradient boosting using leaf-wise tree growth with early stopping. Faster than XGBoost on large feature sets.
- **Setup:** `n_estimators=5000`, `learning_rate=0.005`, `num_leaves=10`, `max_depth=4`, `early_stopping(150 rounds)`
- **Result:** Train: 94.18% | Val: 89.09% | Test: 88.9% | F1: 0.9059 | Gap: 5.3% — Balanced
- **Strength:** Strong regularization prevents overfitting; good balance of speed and accuracy

- **Note:** Did not trigger early stopping — ran full 5000 rounds. Best iteration: 4995

15.10 CatBoost

- **Concept:** Gradient boosting with symmetric trees and built-in handling of categorical features.
- **Hyperparameter Tuning:** iterations $\in \{500, 1000, 1500\}$, learning_rate $\in \{0.01, 0.05, 0.1\}$.
Best: iterations=500, lr=0.01
- **Result:** Train Acc: 92.71% | Val Acc: 92.58% | F1: 0.9088 | Status: Balanced
- **Strength:** Smallest train/val gap among boosting models (0.13%) — most stable ensemble classifier

16. Hyperparameter Tuning

Hyperparameter tuning was applied to 5 models using GridSearchCV (3-Fold CV) and to XGBoost using random search (15 trials). CatBoost was tuned manually by varying iterations and learning_rate systematically.

16.1 SVM — C and gamma

C controls the penalty for misclassification. gamma controls the influence radius of each training sample.

C Value	gamma	Effect on Performance
C=0.1	gamma=scale	Lower accuracy — underfits, too much regularization
C=1.0	gamma=scale	Moderate accuracy — baseline performance
C=10.0	gamma=0.01	Best: Val Acc ~90.07% — optimal balance of margin and fit

16.2 Decision Tree — max_depth and min_samples_leaf

max_depth controls tree complexity. min_samples_leaf prevents overly specific leaf nodes.

max_depth	min_samples_leaf	Effect on Performance
max_depth=6	min_samples_leaf=20	Underfitting — too constrained, high bias
max_depth=8	min_samples_leaf=10	Moderate performance — better generalisation
max_depth=12	min_samples_leaf=5	Best: 84.82% Val Acc — deeper splits capture more patterns

16.3 Random Forest — n_estimators and max_depth

More estimators reduce variance. Deeper trees capture more complex patterns but risk overfitting.

n_estimators	max_depth	Effect on Performance
n_estimators=100	max_depth=10	Faster but lower accuracy — underfitting
n_estimators=200	max_depth=15	Good balance — moderate accuracy
n_estimators=300	max_depth=20	Best: 88.80% Val Acc — more trees improve stability

16.4 Gradient Boosting — n_estimators, learning_rate, max_depth

Lower learning rate with more estimators generally improves accuracy but increases training time.

n_estimators	lr / max_depth	Effect on Performance
--------------	----------------	-----------------------

n_estimators=150	lr=0.05, max_depth=3	Best: 92.55% Val Acc — controlled complexity
n_estimators=200	lr=0.05, max_depth=3	Marginal improvement — slight overfitting tendency
n_estimators=150	lr=0.1, max_depth=4	Faster convergence but lower generalisation

16.5 CatBoost — iterations and learning_rate

More iterations with low learning rate yields the most stable training curve.

Iterations	Learning Rate	Effect on Performance
500	0.01	Best: Train 92.71% / Val 92.58% — smallest gap, most stable
1000	0.01	Slight improvement in train but val plateaus — marginal gain
500	0.05	Faster convergence but slightly higher train/val gap
500	0.1	Fastest convergence — risk of overshooting optimal solution

Overall Tuning Finding: Across all models, moderate complexity with regularization consistently outperformed extreme settings. Very high C in SVM and very deep trees in RF/DT showed diminishing returns or signs of near-overfitting.

17. Model Results & Bar Graphs

17.1 Accuracy Comparison

The bar chart below compares the validation accuracy of all 10 classifiers. XGBoost achieved the highest accuracy at 92.9%, followed by CatBoost (92.6%) and Gradient Boosting (92.6%).



Figure 6 — Comparison of Accuracy (%) across all 10 classifiers on the Validation Set

17.2 Training Time Comparison

SVM required the longest training time (20.44s) due to its quadratic complexity with the RBF kernel. KNN and Decision Tree were the fastest (near 0s and 0.08s respectively) since they require minimal computation during fit.



Figure 7 — Total Training Time (seconds) per classifier

17.3 Test Time Comparison

LightGBM had the longest test/inference time (1.50s) due to the large number of trees (5000 estimators). Logistic Regression was the fastest at effectively 0s. SVM also showed relatively high test time (0.62s) due to kernel computation on the full feature set.

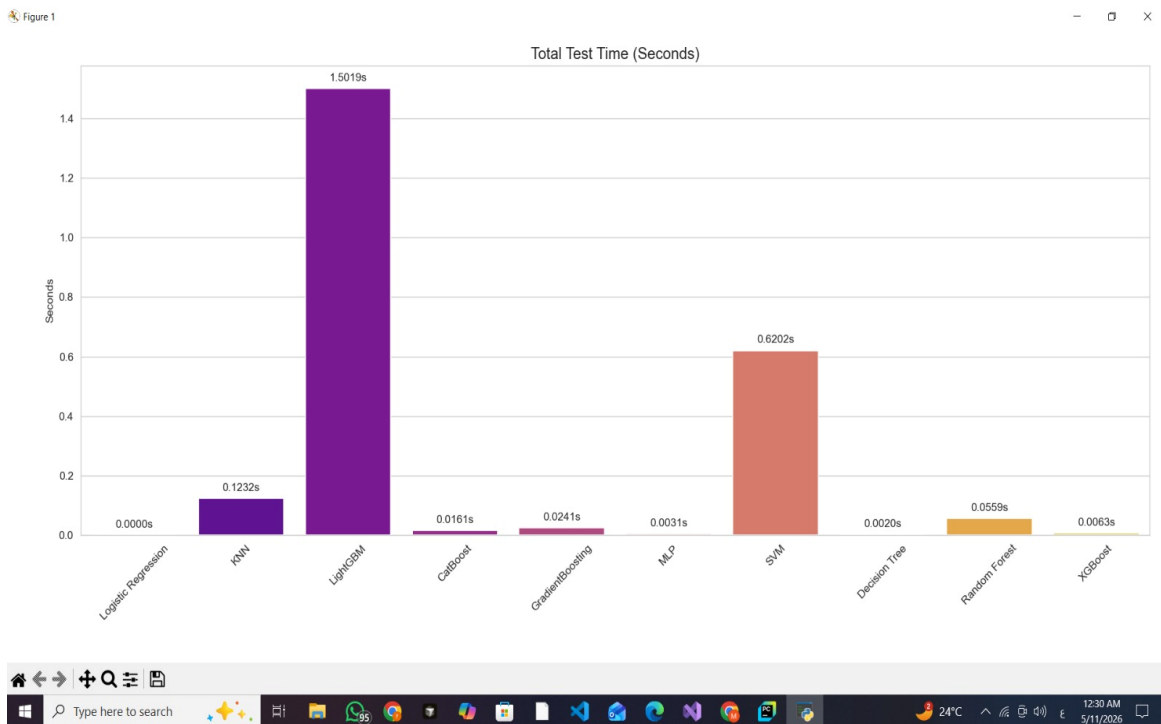


Figure 8 — Total Test/Inference Time (seconds) per classifier

17.4 Final Comparison Table

The table below provides a complete summary of all models ranked by validation accuracy:

Model	Train Acc	Val Acc	F1 (weighted)	Status	Best Params
XGBoost	97.71%	92.90%	0.9206	Balanced	max_depth=4, lr=0.1, subsample=0.7
GradientBoosting	94.31%	92.55%	0.9147	Balanced	n_estimators=150, lr=0.05, max_depth=3
CatBoost	92.71%	92.58%	0.9088	Balanced	iterations=500, lr=0.01
MLP	91.38%	91.05%	0.8930	Balanced	hidden=(128,64,32), relu, adam
SVM	99.33%	90.07%	0.8949	Balanced	C=10.0, gamma=0.01
LightGBM	94.18%	89.09%	0.9059	Balanced	num_leaves=10, max_depth=4, lr=0.005
Random Forest	95.06%	88.80%	0.9018	Balanced	max_depth=20, n_estimators=300
KNN	88.07%	86.78%	0.8324	Balanced	n_neighbors=15, metric=manhattan
Logistic Regression	87.60%	84.93%	0.8774	Balanced	C=0.1, solver=saga
Decision Tree	90.04%	84.82%	0.8755	Balanced	max_depth=12, min_samples_leaf=5

All 10 models achieved Balanced status (train/val gap < 10%). The SVM gap was 9.26% — just within the balanced threshold.

18. Conclusion — Milestone 2

18.1 Summary of Findings

Milestone 2 extended the project from regression to multi-class classification, predicting whether a Steam game falls into the Low, Medium, or High popularity category. Ten classifiers were trained, tuned, and evaluated. Key findings:

- XGBoost achieved the best overall performance with 92.90% validation accuracy and the highest F1 score (0.9206), confirming its superiority as a gradient boosting framework for tabular data.
- All 10 models achieved Balanced status — no model suffered from significant overfitting, validating the preprocessing and regularization strategies used.
- Classification accuracy (84–93%) is remarkably high given that the task predicts a 3-class label with no direct indicator of the target class in the features.
- Ensemble/boosting methods (XGBoost, CatBoost, GradientBoosting) dominated the top 3 positions, consistent with the regression phase findings.
- MLP showed the smallest train/val gap (0.33%) — the most balanced model — while SVM showed the largest gap (9.26%) but still within the balanced threshold.

18.2 Intuitions Proved

- ✓ Proved: Boosting methods outperform single models — XGBoost, CatBoost, and GradientBoosting consistently ranked in the top 3, confirming that ensemble learning is the most effective approach for this dataset.
- ✓ Proved: Feature engineering from Milestone 1 transfers well to classification — the same features (Metacritic_Per_Year, Engagement_Score, Lang_Reach_Score, SteamSpyPlayersVariance) remained informative for predicting popularity tiers.
- ✓ Proved: Class weight balancing was necessary — without `class_weight='balanced'`, linear models (LR, SVM) showed biased predictions toward the majority class.

18.3 Intuitions Disproved or Surprising

- ✗ Disproved: LightGBM was expected to be the fastest model given its leaf-wise growth strategy. However, its 5000-estimator setup made it the slowest at inference (1.50s test time) and second-slowest at training (16.34s).
- ✗ Surprising: KNN with PCA(30 components) achieved 86.78% accuracy — competitive with Random Forest (88.80%). The dimensionality reduction via PCA was crucial; without it, KNN would have been significantly worse.
- ✗ Surprising: CatBoost achieved the smallest train/val gap (0.13%) among all boosting models — more stable than XGBoost despite XGBoost's higher raw accuracy. This suggests CatBoost's symmetric tree structure provides better regularization for this specific dataset.
- ✗ Surprising: Decision Tree, despite being the weakest tree-based model (84.82%), still outperformed Logistic Regression (84.93%) by a tiny margin — suggesting even a single tree captures non-linearities better than a linear classifier.

18.4 Regression vs Classification — Key Takeaways

Comparing both milestones:

- In regression, the best R^2 was 82.25% (Gradient Boosting). In classification, the best accuracy was 92.90% (XGBoost). The classification task appears more learnable from the available features.
 - The same preprocessing pipeline worked for both tasks with minimal modifications — validating the robustness of the feature engineering approach.
 - The shift from a continuous target (regression) to discrete categories (classification) made the problem easier for boosting models, which can create sharp decision boundaries between classes.
-

19. Deployment_Link

Link:

<https://game-popularity-classification-msx6hxtkrxcfq6gfhmhjkh.streamlit.app/>
